



ค่ายโอลิมปิกวิชาการ วิชาคอมพิวเตอร์

ค่าย 2

4. Brute Force and Backtracking Algorithms วิธีแก้ปัญหาแบบตะลุยคำตอบ และแบบย้อนรอย

ปรับปรุงจากเอกสารประกอบการสอนของ อ. ยนต์ชนก เขาแก้ว

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

Outline

- Introduction Brute Force
- Permutations and Combinations
- Example of Brute Force
- Introduction Backtracking Algorithm
- Example of Backtracking Algorithm



วิธีแก้ปัญหาแบบตะลุยคำตอบ (Brute force Algorithm)



Introduction Brute Force

- A straightforward approach, usually based directly on the problem's statement and definitions of the concepts involved
- A brute force solution to a problem involving search for an element with a special property, usually among combinatorial objects such as permutations, combinations, or subsets of a set.
- Example:
 - String Matching
 - Closest pair
 - Traveling Salesman Problem
 - The Assignment Problem



Permutations and Combinations

- How many ways can we select three students from a group of five students to stand in line for a picture?
- How many different committees of three students can be formed from a group of four students?

Permutations

- A permutation of a set of distinct objects is an ordered arrangement of these objects.

- **Permutation**

If n is a positive integer and r is an integer with $1 \leq r \leq n$, then there are r -permutations of a set with n distinct elements.

- **Corollary**

If n and r are integers with $0 \leq r \leq n$, then
$$P(n, r) = \frac{n!}{(n - r)!}$$

Combinations

- Let $S = \{1,2,3,4\}$ then $\{1,3,4\}$ is 3-combination from S . Note that $\{4,1,3\}$ is the same 3-combination as $\{1,3,4\}$, because the order in which the elements of a set are listed does not matter.

- **r-Combinations**

The number of r -combinations of a set with n elements, where n is a nonnegative integer and r is an integer with $0 \leq r \leq n$, equals $C(n,r) = \frac{n!}{r!(n-r)!}$



Brute-Force Strengths and Weaknesses

■ Strengths

- wide applicability
- simplicity
- yields reasonable algorithms for some important problems (e.g., matrix multiplication, sorting, searching, string matching)

■ Weaknesses

- rarely yields efficient algorithms
- some brute-force algorithms are unacceptably slow
- not as constructive as some other design techniques

Brute-Force String Matching

- pattern: a string of m characters to search for
- text: a (longer) string of n characters to search in
- problem: find a substring in the text that matches the pattern

Brute-force algorithm

Step 1 Align pattern at beginning of text

Step 2 Moving from left to right, compare each character of pattern to the corresponding character in text until all characters are found to match (successful search); or a mismatch is detected

Step 3 While pattern is not found and the text is not yet exhausted, realign pattern one position to the right and repeat Step 2



Examples of Brute-Force String Matching

1. Pattern: 001011

Text: 10010101101001100101111010

2. Pattern: happy

Text: It is never too late to have a happy
childhood.

Pseudocode

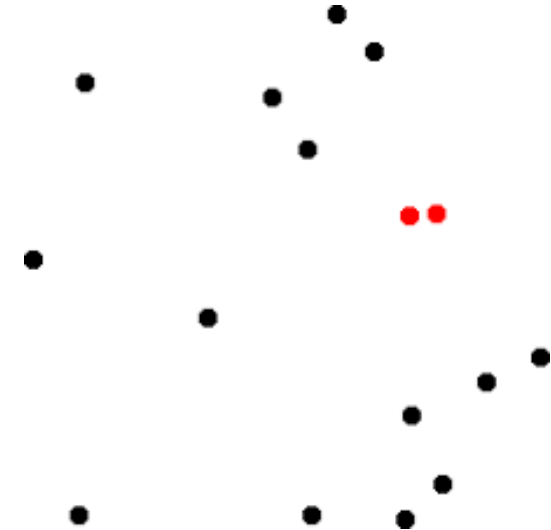
ALGORITHM *BruteForceStringMatch*($T[0..n - 1]$, $P[0..m - 1]$)

//Implements brute-force string matching
//Input: An array $T[0..n - 1]$ of n characters representing a text and
// an array $P[0..m - 1]$ of m characters representing a pattern
//Output: The index of the first character in the text that starts a
// matching substring or -1 if the search is unsuccessful

for $i \leftarrow 0$ **to** $n - m$ **do**
 $j \leftarrow 0$
 while $j < m$ **and** $P[j] = T[i + j]$ **do**
 $j \leftarrow j + 1$
 if $j = m$ **return** i
return -1

Closest-Pair Problem

- Find the two closest points in a set of n points (in the two-dimensional Cartesian plane).



Brute-force algorithm

Compute the distance between every pair of distinct points and return the indexes of the points for which the distance is the smallest.

Closest-Pair Brute-Force Algorithm (cont.)

ALGORITHM *BruteForceClosestPoints(P)*

//Input: A list P of n ($n \geq 2$) points $P_1 = (x_1, y_1), \dots, P_n = (x_n, y_n)$

//Output: Indices $index1$ and $index2$ of the closest pair of points

$dmin \leftarrow \infty$

for $i \leftarrow 1$ **to** $n - 1$ **do**

for $j \leftarrow i + 1$ **to** n **do**

$d \leftarrow \text{sqrt}((x_i - x_j)^2 + (y_i - y_j)^2)$ //*sqrt* is the square root function

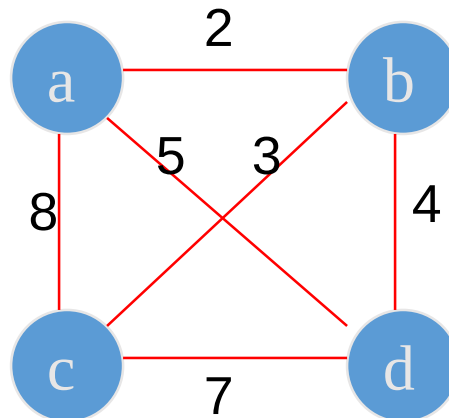
if $d < dmin$

$dmin \leftarrow d; index1 \leftarrow i; index2 \leftarrow j$

return $index1, index2$

Traveling Salesman Problem

- Given n cities with known distances between each pair, find the shortest tour that passes through all the cities exactly once before returning to the starting city
- Alternatively: Find shortest *Hamiltonian circuit* in a weighted connected graph
- Example:



TSP Solution

Tour

$a \rightarrow b \rightarrow c \rightarrow d \rightarrow a$

$a \rightarrow b \rightarrow d \rightarrow c \rightarrow a$

$a \rightarrow c \rightarrow b \rightarrow d \rightarrow a$

$a \rightarrow c \rightarrow d \rightarrow b \rightarrow a$

$a \rightarrow d \rightarrow b \rightarrow c \rightarrow a$

$a \rightarrow d \rightarrow c \rightarrow b \rightarrow a$

More tours?

Cost

$$2+3+7+5 = 17$$

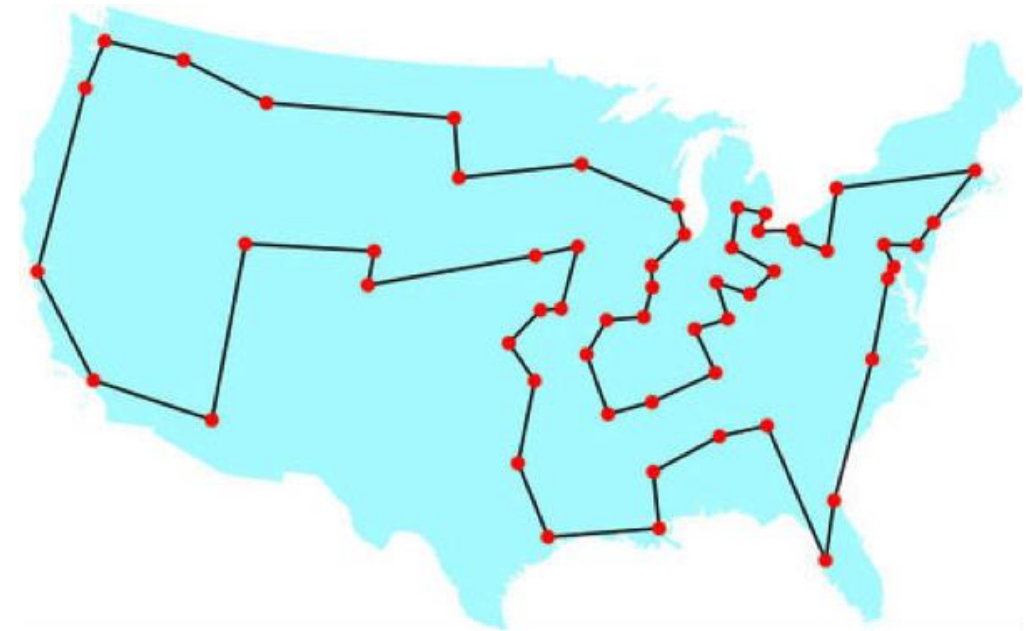
$$2+4+7+8 = 21$$

$$8+3+4+5 = 20$$

$$8+7+4+2 = 21$$

$$5+4+3+8 = 20$$

$$5+7+3+2 = 17$$



Less tours?

The Assignment Problem

- There are n people who need to be assigned to n jobs, one person per job. The cost of assigning person i to job j is $C[i,j]$. Find an assignment that minimizes the total cost.

	Job 0	Job 1	Job 2	Job 3
Person 0	9	2	7	8
Person 1	6	4	3	7
Person 2	5	8	1	8
Person 3	7	6	9	4

- Algorithmic Plan: Generate all legitimate assignments, compute their costs, and select the cheapest one.
- How many assignments are there? Pose the problem as the one about a cost matrix:

Assignment Problem by Exhaustive Search

9 2 7 8
 6 4 3 7
 $C =$ 5 8 1 8
 7 6 9 4

Assignment (col.#s)

1, 2, 3, 4

1, 2, 4, 3

1, 3, 2, 4

1, 3, 4, 2

1, 4, 2, 3

1, 4, 3, 2

Total Cost

$9+4+1+4=18$

$9+4+8+9=30$

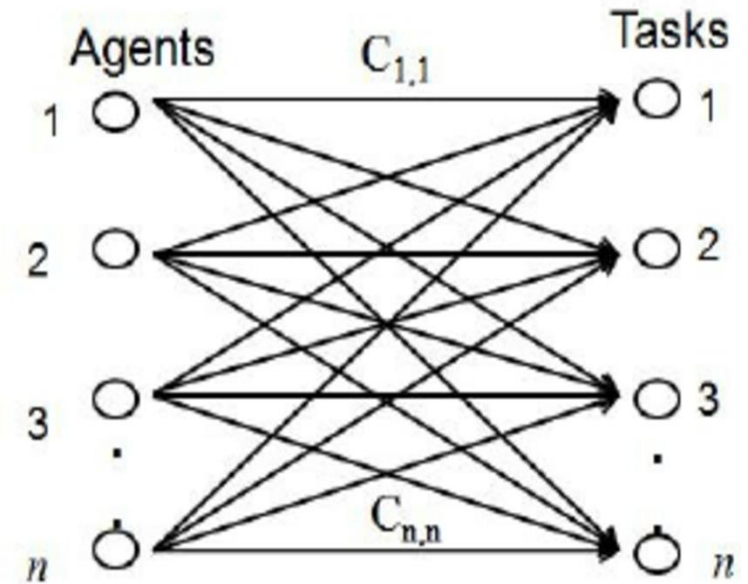
$9+3+8+4=24$

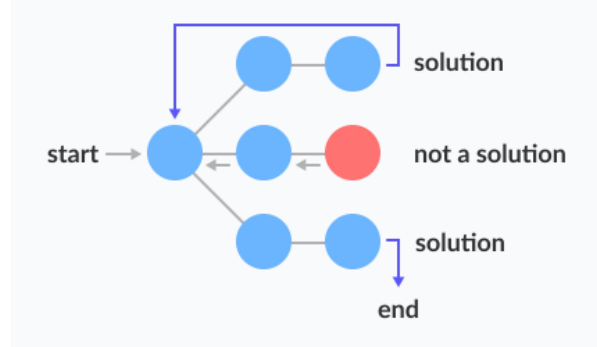
$9+3+8+6=26$

$9+7+8+9=33$

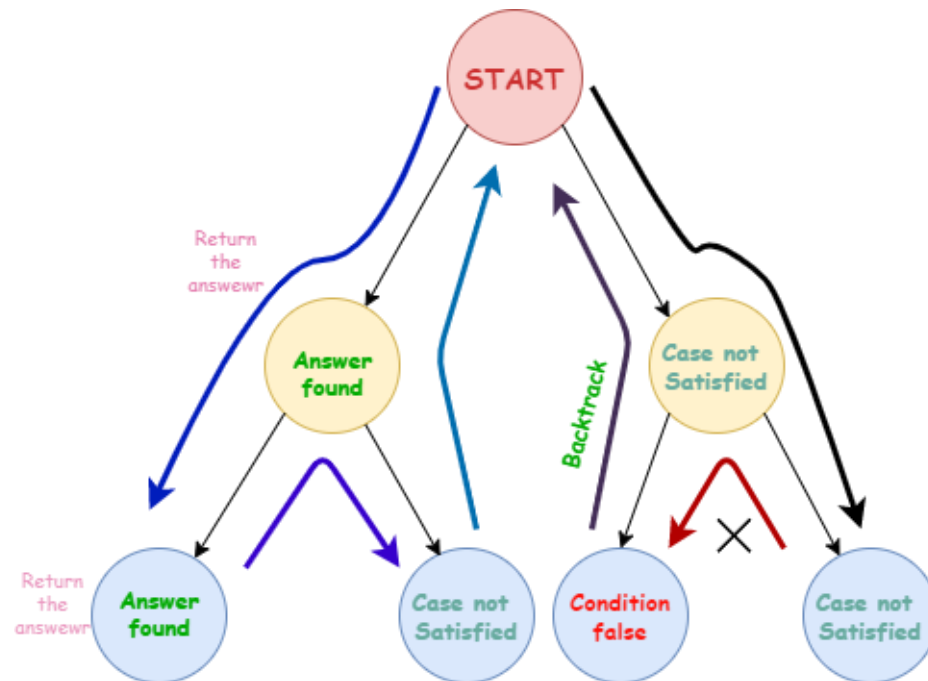
$9+7+1+6=23$

etc.





วิธีแก้ปัญหาแบบย้อนรอย (Backtracking Algorithm)



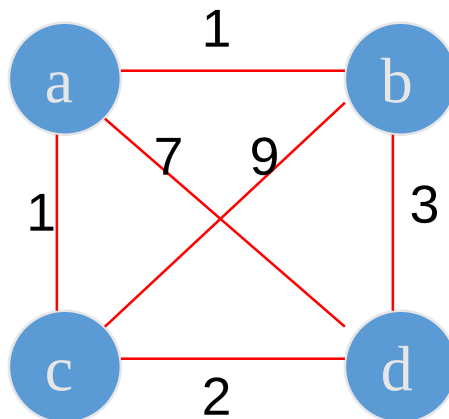


Introduction Backtracking Algorithm

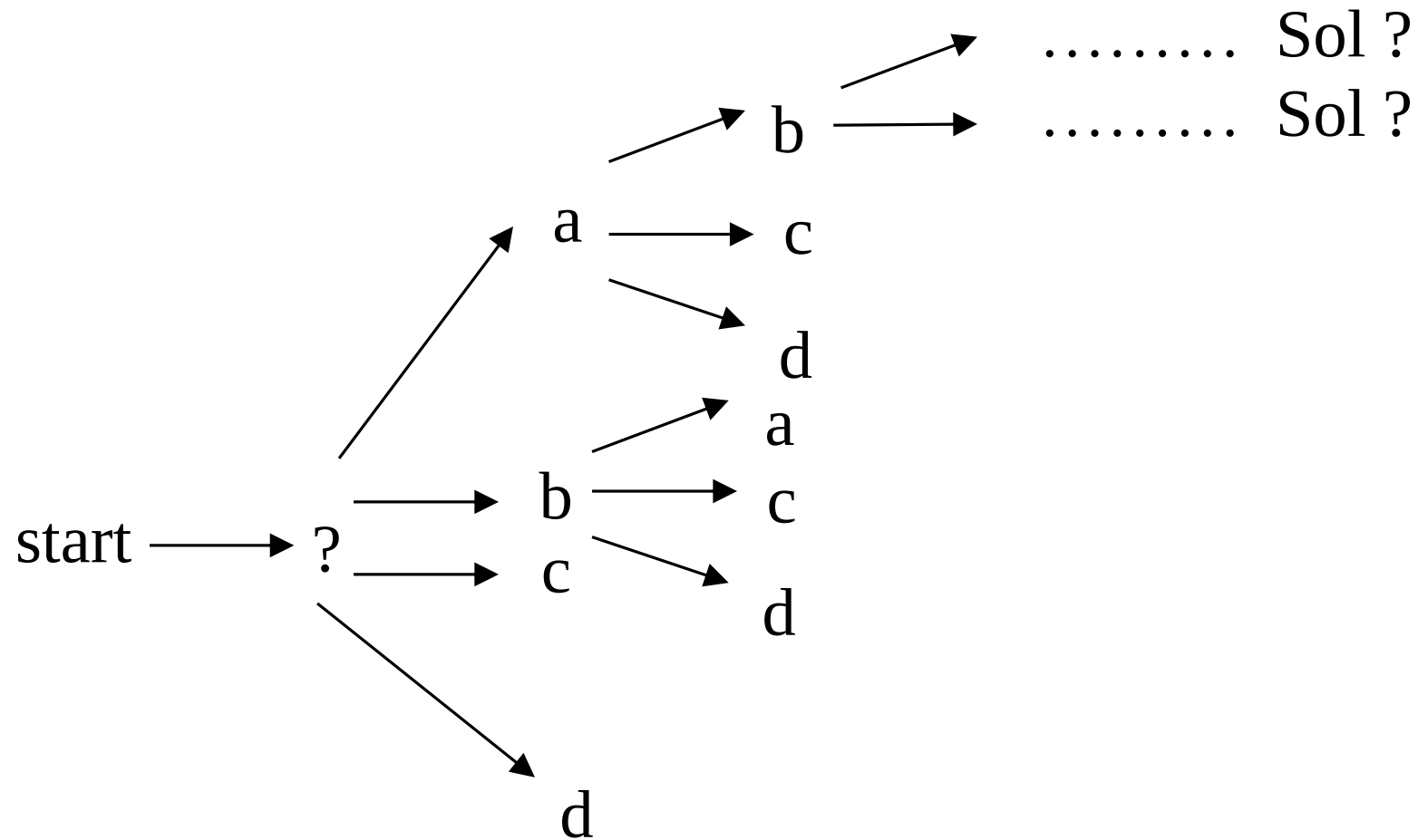
- Suppose you have to make a series of *decisions*, among various *choices*, where
 - You don't have enough information to know what to choose
 - Each decision leads to a new set of choices
 - Some sequence of choices (possibly more than one) may be a solution to your problem
- **Backtracking** is a methodical way of trying out various sequences of decisions, until you find one that “works”

Traveling Salesman Problem

- Given n cities with known distances between each pair, find the shortest tour that passes through all the cities exactly once before returning to the starting city
- Alternatively: Find shortest *Hamiltonian circuit* in a weighted connected graph

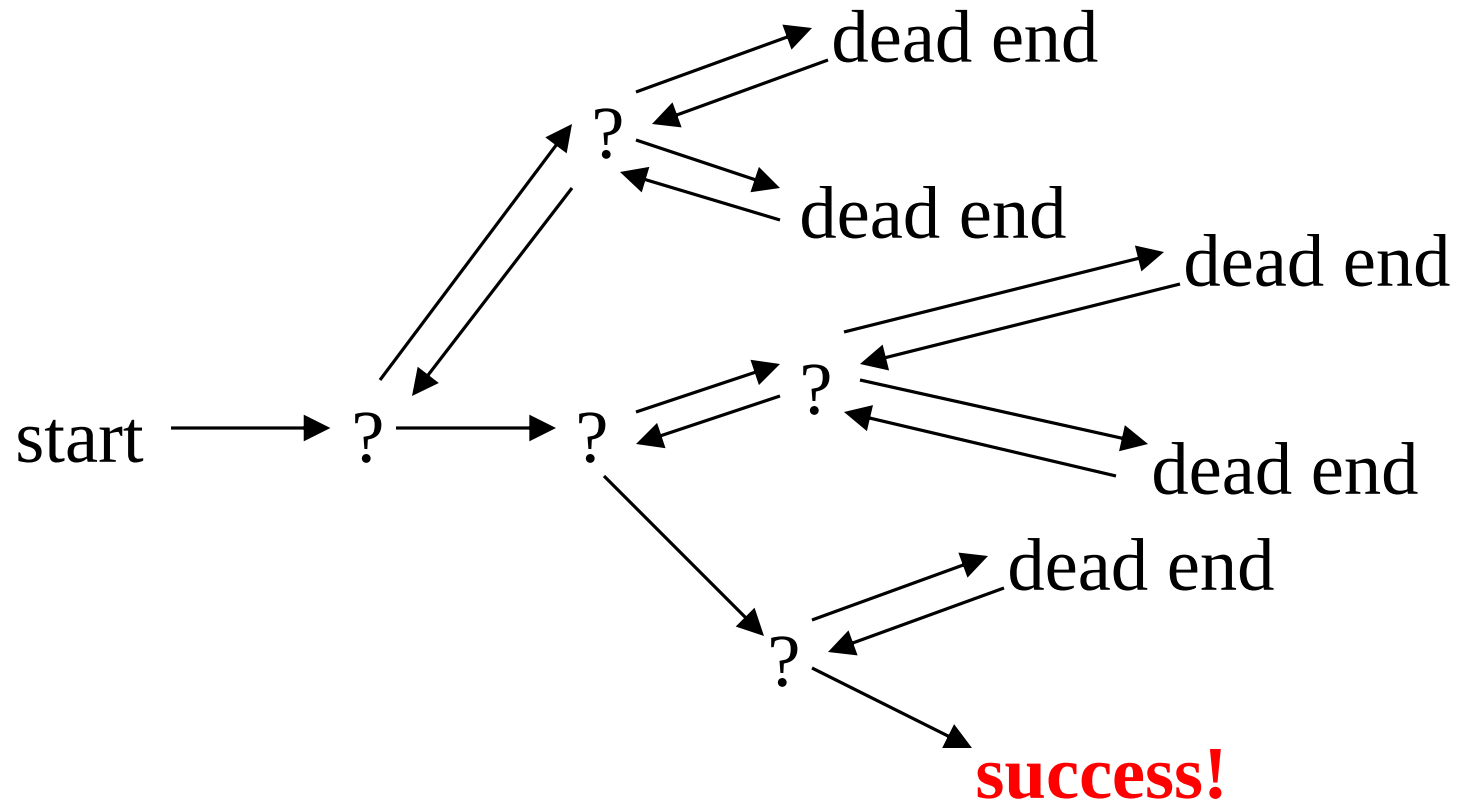


Traveling Salesman Problem



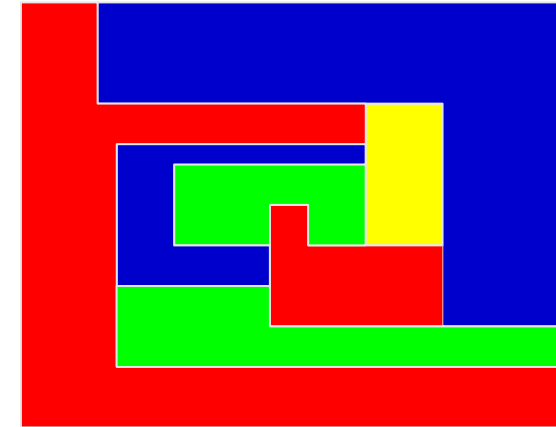


Traveling Salesman Problem (3)



Coloring a map

- You wish to color a map with not more than four colors
 - red, yellow, green, blue
- Adjacent countries must be in different colors
- You don't have enough information to choose colors
- Each choice leads to another set of choices
- One or more sequences of choices may (or may not) lead to a solution
- Many coloring problems can be solved with backtracking



Coloring a map

- The **Four Color Theorem** states that any map on a plane can be colored with no more than four colors, so that no two countries with a common border are the same color
- For most maps, finding a legal coloring is easy
- For some maps, it can be fairly difficult to find a legal coloring
- We will develop a complete Java program to solve this problem

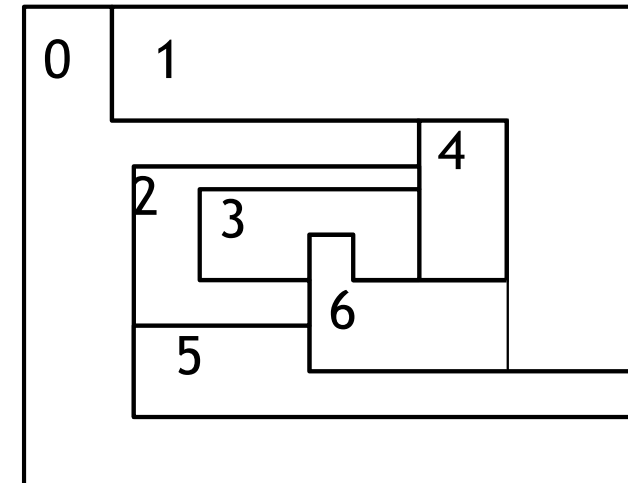
Coloring a map Data structures

- We need a data structure that is easy to work with, and supports:
 - Setting a color for each country
 - For each country, finding all adjacent countries
- We can do this with two arrays
 - An array of “colors”, where `countryColor[i]` is the color of the i^{th} country
 - A ragged array of adjacent countries, where `map[i][j]` is the j^{th} country adjacent to country i
 - Example: `map[5][3]==8` means the 3^{th} country adjacent to country 5 is country 8

Coloring a map

■ `int map[][];`

```
void createMap() {
    map = new int[7][];
    map[0] = new int[] { 1, 4, 2, 5 };
    map[1] = new int[] { 0, 4, 6, 5 };
    map[2] = new int[] { 0, 4, 3, 6, 5 };
    map[3] = new int[] { 2, 4, 6 };
    map[4] = new int[] { 0, 1, 6, 3, 2 };
    map[5] = new int[] { 2, 6, 1, 0 };
    map[6] = new int[] { 2, 3, 4, 1, 5 };
}
```



Coloring a map

```
■ static final int NONE = 0;  
  static final int RED = 1;  
  static final int YELLOW = 2;  
  static final int GREEN = 3;  
  static final int BLUE = 4;
```

```
int mapColors[] = { NONE, NONE, NONE, NONE, NONE, NONE, NONE };
```

The main program

(The name of the enclosing class is **ColoredMap**)

```
public static void main(String args[]) {  
    ColoredMap m = new ColoredMap();  
    m.createMap();  
    boolean result = m.explore(0, RED);  
    System.out.println(result);  
    m.printMap();  
}
```

Coloring a map: the backtracking method

```
boolean explore(int country, int color) {  
    if (country >= map.length) return true;  
    if (okToColor(country, color)) {  
        mapColors[country] = color;  
        for (int i = RED; i <= BLUE; i++) {  
            if (explore(country + 1, i)) return true;  
        }  
    }  
    return false;  
}
```



Coloring a map: checking if a color can be used

```
boolean okToColor(int country, int color) {  
    for (int i = 0; i < map[country].length; i++) {  
        int ithAdjCountry = map[country][i];  
        if (mapColors[ithAdjCountry] == color) {  
            return false;  
        }  
    }  
    return true;  
}
```

Coloring a map

```
void printMap() {  
    for (int i = 0; i < mapColors.length; i++) {  
        System.out.print("map[" + i + "] is ");  
        switch (mapColors[i]) {  
            case NONE:    System.out.println("none");    break;  
            case RED:     System.out.println("red");     break;  
            case YELLOW:  System.out.println("yellow");  break;  
            case GREEN:   System.out.println("green");   break;  
            case BLUE:    System.out.println("blue");    break;  
        }  
    }  
}
```

Coloring a map: recap

- We went through all the countries recursively, starting with country zero
- At each country we had to decide a color
 - It had to be different from all adjacent countries
 - If we could not find a legal color, we reported failure
 - If we could find a color, we used it and recurred with the next country
 - If we ran out of countries (colored them all), we reported success
- When we returned from the topmost call, we were done

Min Coin Change problem

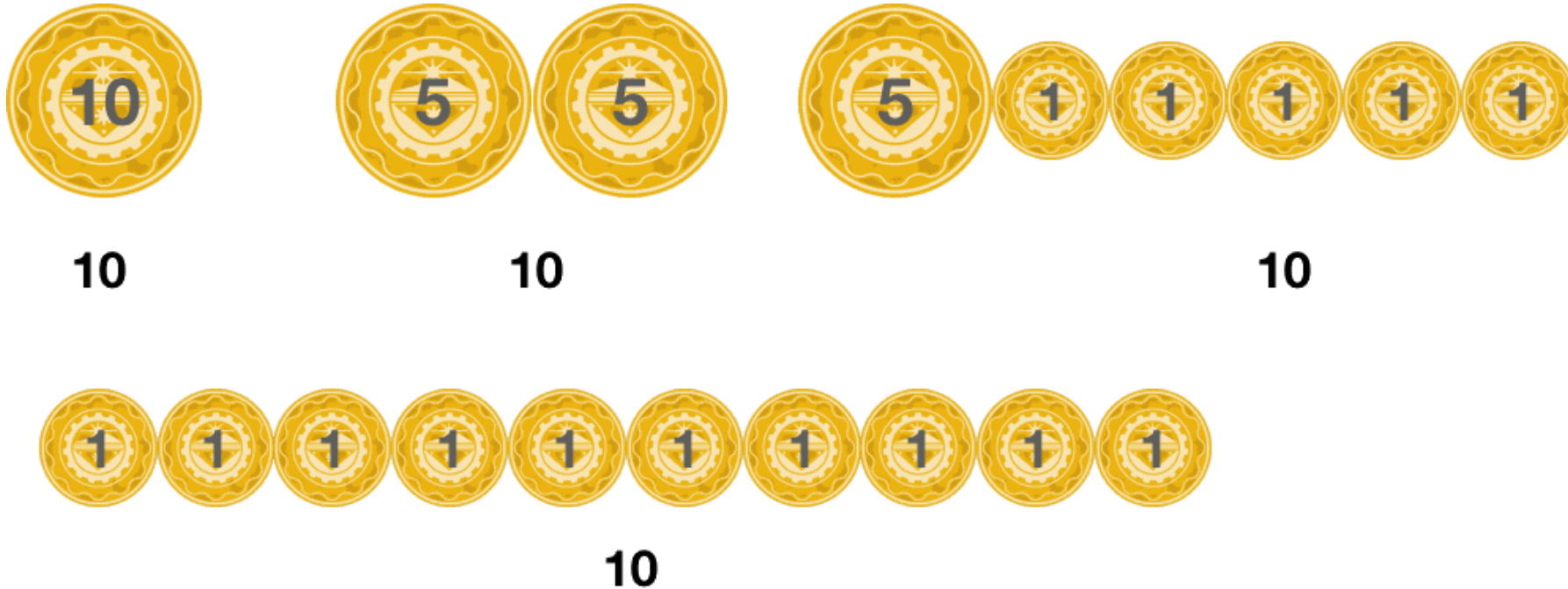
- Coin changing problem
 - จำนวนเงินที่ต้องการทอน : n บาท
 - ประเภทของเหรียญที่มีในท้องตลาด : 10, 5, 2, 1
 - วิธีการที่จะใช้จำนวนเหรียญที่น้อยที่สุดคืออะไร ?
- i.e. $n = 10a + 5b + 2c + d$, what are the a , b , c , and d , minimizing $(a+b+c+d)$
- จงออกแบบอัลกอริธึมสำหรับแก้ปัญหาลักษณะนี้ ?

Min Coin Change problem

- $n = 10a + 5b + 2c + d$, what are the a , b , c , and d , minimizing $(a+b+c+d)$
- ลองทำแบบ brute-force ก่อน ?
- เราต้องการทอนเงิน n บาท
- Try all the combinations where $a \leq n$, $b \leq n$, $c \leq n$, $d \leq n$
- Choose all the combinations that $n = 25a + 10b + 5c + d$
- Choose the combination with smallest $(a+b+c+d)$

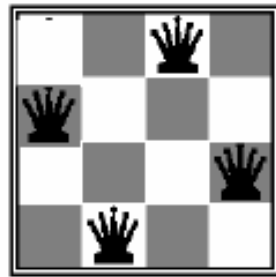
Min Coin Change problem

- ลองประยุกต์ Backtracking Algorithm มาใช้ลดจำนวนการทำงานจาก Brute force ดู?

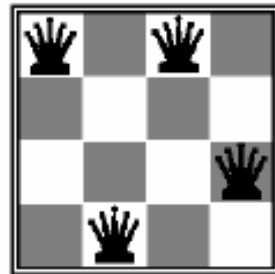


The N-Queens Problem

- Find a configuration of n queens not attacking each other



Goal configuration



Bad goal configuration

The N-Queens Problem

- นิยามของปัญหา ปัญหานี้เป็นการวางควีนจำนวน 8 ตัวลงในกระดานหมากรุกขนาด 8×8 โดยควีนทั้งหมดต้องไม่ สามารถ “กิน” ควีนตัวอื่นได้ โดยควีนของหมากรุกฝรั่งสามารถ “กิน” หมากรุกตัวอื่นได้ (รวมทั้งควีนตัวอื่นด้วย) หากหมากตัวนั้นอยู่ใน
 ๑. แถวเดียวกัน
 ๒. คอลัมน์เดียวกัน
 ๓. แนวเส้น ทแยงมุมเดียวกัน (แนวเส้นทแยงมุม มีสองแนว)

The N-Queens Problem

- แนวทางแก้ปัญหา การแก้ปัญหาเริ่มต้นด้วยแนวคิดที่ว่าเมื่อมีการวางควีนแต่ละตัวลงในกระดาน ควีนตัวนั้นจะมีการคุม ตำแหน่งอื่นๆในแถว คอลัมน์ และแนวทะแยงเดียวกับควีนนั้น ตำแหน่งเหล่านี้ไม่สามารถวางควีนตัวอื่นได้ ดังนั้นเมื่อวางควีนแต่ละตัวลงไปแล้ว จึงต้องตัดสินใจว่า ตำแหน่งใดอยู่ใต้การคุมของควีน และ ตำแหน่งใดเป็นอิสระ เพื่อใช้ตัดสินใจในการวางควีนตัวต่อไป หากไม่สามารถวางควีนได้ จะทำการถอนควีนที่วางไว้แล้วออกทีละตัว ทดลองวางควีนตัวใหม่หากทำได้จะดำเนินการต่อไป หากไม่สามารถทำได้จะทำการถอนควีนตัวที่วาง ก่อนหน้านี้ ออกอีก จนจะสามารถวางควีน ได้ตามเงื่อนไข ซึ่งลักษณะการดำเนินการเช่นนี้เหมาะสมกับการทำงานแบบ recursive โดยมีแนวคิดเบื้องต้นดังนี้

The N-Queens Problem

เงื่อนไขก่อนการทำงาน - กำหนดให้จำนวนควีนบนกระดาน, n , มีค่าเริ่มต้นเป็น 0

begin

```
for ตำแหน่ง  $p$  ใดๆทุกตำแหน่งที่ไม่ถูกคุมโดยควีนที่มีอยู่แล้วบนกระดาน do  
    วางควีนลงที่ตำแหน่ง  $p$   
    เพิ่มจำนวนควีนที่อยู่บนกระดาน ( $n++$ )  
    if จำนวนควีน = 8 then  
        พิมพ์รูปแบบการวางควีนบนกระดาน  
    else  
        เรียกตัวเองซ้ำเพื่อวางควีนตัวต่อไป  
    end {if}  
    ถอนควีนออกจากตำแหน่ง  $p$   
    ลดจำนวนควีนที่อยู่บนกระดาน ( $n--$ )  
end {for}
```

end

The N-Queens Problem

เงื่อนไขก่อนการทำงาน - กำหนดให้จำนวนควีนบนกระดาน, n , มีค่าเริ่มต้นเป็น 0

begin

```
for ตำแหน่ง  $p$  ใดๆทุกตำแหน่งที่ไม่ถูกคุมโดยควีนที่มีอยู่แล้วบนกระดาน do  
    วางควีนลงที่ตำแหน่ง  $p$   
    เพิ่มจำนวนควีนที่อยู่บนกระดาน ( $n++$ )  
    if จำนวนควีน = 8 then  
        พิมพ์รูปแบบการวางควีนบนกระดาน  
    else  
        เรียกตัวเองซ้ำเพื่อวางควีนตัวต่อไป  
    end {if}  
    ถอนควีนออกจากตำแหน่ง  $p$   
    ลดจำนวนควีนที่อยู่บนกระดาน ( $n--$ )  
end {for}
```

end

8-Queens Problem: 12 Unique Solutions

